

DIA 3 — Como Buscar Qualquer Informação Usando SQL

Introdução

Até agora entendemos o que é um banco de dados, como ele é modelado e como o PostgreSQL utiliza memória para entregar um bom desempenho.

Chegou o momento de aprender a linguagem responsável por consultar essas informações: o **SQL (Structured Query Language)**.

O SQL é a linguagem padrão utilizada pelos principais Sistemas Gerenciadores de Banco de Dados (SGBDs), como PostgreSQL, MySQL, SQL Server, Oracle e diversos outros.

Neste capítulo vamos aprender os fundamentos das consultas SQL utilizando exemplos práticos. Nosso objetivo é compreender como localizar informações em tabelas, ordenar resultados, limitar registros, realizar cálculos e preparar o caminho para consultas muito mais complexas.

Para acompanhar todos os exemplos, você pode utilizar gratuitamente o playground do PostgreSQL disponível em:

<https://www.pgtutorial.com/playground/>

Preparando o Ambiente

Antes de realizar qualquer consulta, precisamos criar algumas tabelas e inserir dados de exemplo.

Utilizaremos um pequeno sistema composto por três entidades:

- Clientes;
- Produtos;
- Pedidos.

A tabela **clientes** armazenará os dados das pessoas cadastradas.

A tabela **produtos** armazenará os itens disponíveis para venda.

Já a tabela **pedidos** fará a ligação entre clientes e produtos, registrando quais produtos cada cliente comprou.

Criando as tabelas

```
CREATE TABLE clientes (  
    cod_cliente SERIAL PRIMARY KEY,
```

```

    nome_cliente VARCHAR(50) NOT NULL,
    sobrenome_cliente VARCHAR(50) NOT NULL
);

CREATE TABLE produtos (
    cod_produto SERIAL PRIMARY KEY,
    nome_produto VARCHAR(50) NOT NULL,
    descricao TEXT,
    preco NUMERIC(10,2) CHECK(preco > 0),
    qtde_estoque SMALLINT DEFAULT 0
);

CREATE TABLE pedidos (
    cod_pedido SERIAL PRIMARY KEY,
    cod_cliente INT REFERENCES clientes(cod_cliente),
    cod_produto INT REFERENCES produtos(cod_produto),
    qtde SMALLINT NOT NULL
);

```

Observe que a tabela **pedidos** possui duas chaves estrangeiras (**FOREIGN KEY**), responsáveis por relacionar um pedido com um cliente e um produto.

Esse relacionamento será muito importante quando estudarmos **JOINS**.

Inserindo dados

Agora vamos popular nosso banco de dados com algumas informações.

Primeiramente inserimos cinco clientes.

```

INSERT INTO clientes (nome_cliente, sobrenome_cliente)
VALUES ('João', 'Silva');

INSERT INTO clientes (nome_cliente, sobrenome_cliente)
VALUES ('Maria', 'Oliveira');

INSERT INTO clientes (nome_cliente, sobrenome_cliente)
VALUES ('Pedro', 'Santos');

INSERT INTO clientes (nome_cliente, sobrenome_cliente)
VALUES ('Ana', 'Carvalho');

INSERT INTO clientes (nome_cliente, sobrenome_cliente)
VALUES ('Lucas', 'Moraes');

```

Depois cadastramos alguns produtos.

```
INSERT INTO produtos (nome_produto, descricao, preco, qtde_estoque)
VALUES ('Pizza Margherita', 'Pizza tradicional com molho de tomate e
manjericão', 45.00, 10);

INSERT INTO produtos (nome_produto, descricao, preco, qtde_estoque)
VALUES ('Refrigerante Lata', 'Lata 350ml', 6.00, 50);

INSERT INTO produtos (nome_produto, descricao, preco, qtde_estoque)
VALUES ('Hambúrguer', 'Hambúrguer artesanal', 30.00, 20);

INSERT INTO produtos (nome_produto, descricao, preco, qtde_estoque)
VALUES ('Batata Frita', 'Porção de batata frita', 18.00, 15);

INSERT INTO produtos (nome_produto, descricao, preco, qtde_estoque)
VALUES ('Cerveja', 'Long Neck', 8.00, 25);
```

Por fim, registramos alguns pedidos.

```
INSERT INTO pedidos (cod_cliente, cod_produto, qtde)
VALUES (1, 1, 2);

INSERT INTO pedidos (cod_cliente, cod_produto, qtde)
VALUES (2, 3, 1);

INSERT INTO pedidos (cod_cliente, cod_produto, qtde)
VALUES (1, 2, 3);

INSERT INTO pedidos (cod_cliente, cod_produto, qtde)
VALUES (3, 5, 4);

INSERT INTO pedidos (cod_cliente, cod_produto, qtde)
VALUES (4, 4, 1);
```

Com esse pequeno banco de dados já podemos começar a aprender SQL.

A Instrução SELECT

A instrução **SELECT** é a operação mais utilizada em SQL.

Ela é responsável por recuperar informações armazenadas em uma ou mais tabelas.

Sua sintaxe básica é bastante simples.

```
SELECT coluna(s)
FROM tabela;
```

À medida que avançamos, podemos combinar o `SELECT` com filtros, ordenações, junções e diversas outras cláusulas.

```
SELECT coluna(s)
FROM tabela;
```

```
SELECT coluna(s)
FROM tabela
WHERE condição;
```

```
SELECT coluna(s)
FROM tabelas
JOIN ...
```

```
SELECT coluna(s)
FROM tabela
JOIN ...
WHERE condição;
```

Primeiras Consultas

Podemos começar consultando todos os registros de uma tabela.

```
SELECT * FROM clientes;
```

O caractere `*` indica que todas as colunas devem ser retornadas.

Também podemos selecionar apenas algumas colunas.

```
SELECT nome_cliente
FROM clientes;
```

Ou selecionar múltiplas colunas.

```
SELECT nome_cliente,
       sobrenome_cliente
FROM clientes;
```

A ordem das colunas também pode ser alterada.

```
SELECT sobrenome_cliente,  
       nome_cliente  
FROM clientes;
```

Da mesma forma, podemos consultar outras tabelas.

```
SELECT nome_produto  
FROM produtos;  
  
SELECT nome_produto,  
       preco  
FROM produtos;  
  
SELECT *  
FROM pedidos;
```

Perceba que o SQL retorna exatamente as colunas solicitadas.

Quanto menos informações forem recuperadas, menor será a quantidade de dados transferidos entre o banco e a aplicação.

Ordenando Resultados com ORDER BY

Nem sempre desejamos visualizar os registros na ordem em que foram inseridos.

Para isso utilizamos a cláusula **ORDER BY**.

Sua função é ordenar os resultados de acordo com uma ou mais colunas.

Antes dos exemplos, vamos inserir mais alguns produtos.

```
INSERT INTO produtos (cod_produto, nome_produto, descricao, preco,  
qtde_estoque)  
VALUES (6, 'Detergente', 'Detergente líquido 500 ml', 1.89, 32);  
  
INSERT INTO produtos (cod_produto, nome_produto, descricao, preco,  
qtde_estoque)  
VALUES (7, 'Leite Integral', 'Leite integral caixa 1 litro', 6.90, 70);  
  
INSERT INTO produtos (cod_produto, nome_produto, descricao, preco,  
qtde_estoque)  
VALUES (8, 'Refrigerante', 'Garrafa 600 ml', 4.10, 14);  
  
INSERT INTO produtos (cod_produto, nome_produto, descricao, preco,  
qtde_estoque)
```

```
VALUES (9, 'Refrigerante', 'Garrafa 1 litro', 50.00, 16);

INSERT INTO produtos (cod_produto, nome_produto, descricao, preco,
qtde_estoque)
VALUES (10, 'Refrigerante', 'Lata 350 ml', 2.99, 45);

INSERT INTO produtos (cod_produto, nome_produto, descricao, preco,
qtde_estoque)
VALUES (11, 'Suco Integral', 'Suco Integral Caixa', 7.00, 100);

INSERT INTO produtos (cod_produto, nome_produto, descricao, preco,
qtde_estoque)
VALUES (12, 'Margarina', NULL, 11.00, 70);
```

Agora podemos ordenar os resultados.

```
SELECT *
FROM produtos
ORDER BY nome_produto;
```

Por padrão, a ordenação ocorre em ordem crescente (`ASC`).

Para ordenar do maior para o menor utilizamos `DESC` .

```
SELECT *
FROM produtos
ORDER BY qtde_estoque DESC;
```

Também podemos ordenar por mais de uma coluna.

```
SELECT nome_produto,
       preco
FROM produtos
ORDER BY nome_produto,
       preco;
```

Quando existem valores nulos, o PostgreSQL permite controlar sua posição.

```
SELECT nome_produto,
       descricao
FROM produtos
ORDER BY descricao NULLS FIRST;
```

ou

```
SELECT nome_produto,  
       descricao  
FROM produtos  
ORDER BY descricao NULLS LAST;
```

LIMIT e OFFSET

Em muitas situações não precisamos retornar todos os registros da consulta.

Para isso utilizamos as cláusulas `LIMIT` e `OFFSET`.

O `LIMIT` define quantas linhas serão retornadas.

```
SELECT *  
FROM produtos  
ORDER BY preco  
LIMIT 4;
```

Por exemplo, para visualizar os três produtos mais caros.

```
SELECT *  
FROM produtos  
ORDER BY preco DESC  
LIMIT 3;
```

Já o `OFFSET` informa quantas linhas devem ser ignoradas antes do início da contagem.

Se quisermos visualizar o terceiro, quarto e quinto produtos mais caros:

```
SELECT *  
FROM produtos  
ORDER BY preco DESC  
LIMIT 3 OFFSET 2;
```

Também podemos utilizar apenas o `OFFSET`.

```
SELECT *  
FROM produtos  
ORDER BY preco DESC  
OFFSET 2;
```

Essas cláusulas são amplamente utilizadas na implementação de paginação em aplicações web.

Funções de Agregação

Nem sempre queremos listar registros individualmente.

Em muitos casos desejamos calcular estatísticas sobre um conjunto de dados.

Para isso utilizamos as funções de agregação.

COUNT

Conta quantos registros existem.

```
SELECT COUNT(*)  
FROM produtos;
```

Também podemos contar apenas valores distintos.

```
SELECT COUNT(DISTINCT nome_produto)  
FROM produtos;
```

Ou aplicar filtros.

```
SELECT COUNT(*)  
FROM produtos  
WHERE preco >= 10;
```

AVG

Calcula a média dos valores.

```
SELECT AVG(preco)  
FROM produtos;
```

Podemos arredondar o resultado.

```
SELECT ROUND(AVG(preco), 2)  
FROM produtos;
```

MAX e MIN

Retornam respectivamente o maior e o menor valor de uma coluna.

```
SELECT MAX(preco)
FROM produtos;

SELECT MIN(preco)
FROM produtos;
```

SUM

Calcula a soma dos valores.

```
SELECT SUM(preco)
FROM produtos;
```

Um erro bastante comum é tentar combinar funções de agregação com colunas comuns sem realizar agrupamentos.

Por exemplo:

```
SELECT SUM(preco), nome_produto
FROM produtos;
```

Essa consulta gera erro porque o banco não sabe qual nome de produto deve ser associado ao valor total da soma.

Posteriormente estudaremos a cláusula `GROUP BY`, que resolve exatamente esse tipo de situação.

Utilizando Alias com AS

Durante o desenvolvimento de consultas é muito comum desejar nomes mais intuitivos para colunas e tabelas.

Para isso utilizamos **alias**es.

Podemos renomear colunas.

```
SELECT nome_cliente AS Nome,  
       sobrenome_cliente AS Sobrenome  
FROM clientes;
```

Também podemos criar apelidos para tabelas.

```
SELECT c.nome_cliente,  
       c.sobrenome_cliente  
FROM clientes AS c;
```

Os aliases tornam consultas maiores muito mais legíveis e são amplamente utilizados em sistemas profissionais.

Conclusão

Neste capítulo conhecemos os principais fundamentos da linguagem SQL.

Aprendemos como recuperar informações utilizando o `SELECT`, ordenar resultados com `ORDER BY`, limitar registros com `LIMIT` e `OFFSET`, calcular estatísticas utilizando funções de agregação e melhorar a legibilidade das consultas através de aliases.

Esses comandos representam a base da linguagem SQL e serão utilizados constantemente ao longo deste curso.

No próximo capítulo começaremos a explorar filtros com a cláusula `WHERE` e veremos como localizar exatamente as informações desejadas dentro de um banco de dados.